

Record Triggered Mastery

When to Flow and When to Code



Workshop Hosts



Michelle Long
Senior Solutions Architect
Salesforce



Scott Ratliff
Principal Architect Evangelist

Forward Looking Statement

This presentation contains forward-looking statements about, among other things, trend analyses and statements regarding future events, anticipated growth and industry prospects, and our strategies, expectation or plans regarding product releases and enhancements. The achievement or success of the matters covered by such forward-looking statements involves risks, uncertainties and assumptions. If any such risks or uncertainties materialize or if any of the assumptions prove incorrect, results or outcomes could differ materially from those expressed or implied by these forward-looking statements. The risks and uncertainties referred to above include those factors discussed in Salesforce's reports filed from time to time with the Securities and Exchange Commission, including, but not limited to our ability to meet the expectations of our customers; uncertainties regarding AI technologies and their integration into our product offerings; the effect of evolving domestic and foreign government regulations; regulatory developments and regulatory investigations involving us or affecting our industry; our ability to successfully introduce new services and product features, including related to AI and Agentforce; our ability to execute our business plans; the pace of change and innovation and our ability to compete in the markets in which we participate; and our ability to maintain and enhance our brands.

Agenda

1

Introduction

2

Understanding
Automation Density

3

Implementation Best
Practices

4

Design for Scale

5

Governance

6

Q & A

Mindshift

As Salesforce “low code” developers become better / supplemented with AI deciding between Flow v Apex is less about “comfort zone” and more about the Data Model and existing landscape.. Planning for growth while still starting small.

Use Case:

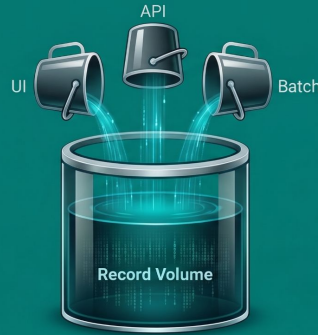
Whenever a commercial Opportunity is marked as 'Closed Won', I need the system to automatically send an email alert to the Onboarding Team, and update the related Account status field to 'Active Customer'

Three Dimensions of Automation Density



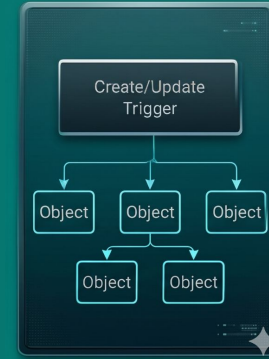
AUTOMATION QUANTITY

Raw number of unique metadata entries (Flows, triggers, etc.) executing during a single DML Event.



RECORD VOLUME

The number of records moving through an object, distinguishing between manual UI edits and massive batch/API loads.



DEPENDENCY SPRAWL

The ripple effect of a save operation measures how many downstream DML updates are triggered across related objects.

Automation Density

Density Level	Automation Quantity	Volume	Sprawl	Architectural Standard
Low Density	< 15	Standard UI	Discrete	Record Triggered Flow
Medium Density	15 – 30	Moderate Batch	Coupled	Hybrid Pattern (Flow + Invocable Apex)
High Density	> 30	Bulk API (10K+)	Complex / Recursive	Apex Triggered Metadata Framework

AI Demo

Use Agentforce Vibes 2.0 to determine density quickly!

Implementation Best Practices

"One Entry Point" Rule of Thumb

- Use a single entry point per Salesforce Object
- The Hybrid Pattern uses Flow for orchestration and Invocable Apex for complex logic.
- Adopt a **Metadata-Driven Trigger Framework** for high-density objects.

Performance Optimization

- **Before-Save** to prevent a second DML operation and recursive save cycles.
- Deduplicate SOQL/Get Records

Safeguards (Recursion & Bypasses)

- Prevent uncontrolled recursion
- Use a **standardized bypass framework** with for large data loads or integration users

Combine these patterns for robust, scalable Salesforce automation

Metadata-Driven Trigger Framework

For high-density objects, use a single Apex trigger delegating to a **Metadata-Driven Trigger Framework**. This allows for declarative control of execution order without constant code deployments.

1. Atomic Classes

Business logic is partitioned into small, highly focused classes (e.g., RecalculateAccountValues) adhering to the **Single Responsibility Principle** for easier testing.

2. Declarative Choreography

Execution order is defined in Custom Metadata (e.g., TriggerAction__mdt). Admins can re-order or remove actions without code deployments.

3. Granular Bypasses

Standardized bypass logic via metadata fields (e.g., BypassPermission__c) allows skipping specific actions during bulk loads or integrations based on user permissions.

4. The Dynamic Dispatcher

A "traffic cop" Apex trigger with no business logic. It calls a central handler (e.g., MetadataTriggerHandler) to query metadata records.

Apex transactional control with declarative, click-to-configure flexibility.

Performance Optimization

Before-Save Updates

- **When to use:** Always use for updates on the record that triggered the automation.
- **Efficiency:** Updates occur in-memory *before* the database commit.
- **Benefit:** Eliminates the need for additional DML operations and prevents recursive save cycles.

Deduplicate SOQL

- **Shared Data:** Avoid redundant queries for the same data point as automation complexity scales.
- **Flow Strategy:** Utilize in-flow variables and pass data to sub-flows or Apex.
- **Caching in Apex:**
 - **Static Variables:** Best for sharing data within a single transaction.
 - **Platform Cache:** Use for high-read, low-change data across multiple user transactions.

Safeguards

Avoid Recursion

Recursion occurs when an automation performs a data update that inadvertently causes that same automation to fire again.

Uncontrolled recursion can cause infinite loops that crash your transactions!

- Prevent uncontrolled recursion by using **comparison logic** instead of current state value (e.g. Status from 'Open' to 'Closed', not just 'Open').
- Compare `$Record` to `$RecordPrior` in Flow or `Trigger.new` to `Trigger.oldMap` in Apex.

Bulk Bypasses

Use a standardized bypass framework with **Custom Permissions** for large data loads or integration users.

- Establish a centralized **Custom Permission** to use when performing bulk data loads on a given object.
- Ensure that Apex and Flows check for this permission before running. If the user has it, **DO NOT** run.
- If using a **Metadata-Driven Trigger Framework**, store this check in your metadata for easy updates.

AI Demo

*Use Agentforce Vibes 2.0 to support Record
Trigger Implementation Best Practices*

Design for Scale



When to Decouple: When synchronous triggers hit governor limits due to heavy computations or massive volumes, work must be offloaded to prevent transaction rollbacks.

Flow Asynchronous

Best for simple "fire and forget" tasks like emails or simple callouts.

Watch out for daily governor limits!

CDC (Change Data Capture)

Highly scalable pattern using event buses and dedicated subscriber triggers for long-running work.

Max 5 Objects & Cannot support state transitions

Scheduled Jobs

When high volume is required but CDC isn't viable, use Scheduled Flows or Apex to process "pending" records.



Queueable Apex from Triggers

Last Resort! Be mindful of risks and limits when enqueueing jobs from triggers during bulk data loads.

Change Data Capture (CDC)

i Change Data Capture (CDC) is a scalable, resilient architectural pattern for handling asynchronous logic in high-volume scenarios. By offloading processing from synchronous triggers to an event bus, CDC improves system performance and reliability.

How It Works

After setup; Salesforce platform automatically publishes a change event. A subscriber Apex trigger then processes this event asynchronously.

Trade-offs & Limits

- **No Prior State:** Lacks `Trigger.oldMap`; logic requiring state transitions requires querying field history.
- **Object Limits:** Default allowed on up to 5 objects; add-on license required for more.

Implementation

- **Enable:** Select objects via Setup or Metadata API.
- **Subscribe:** Write an Apex trigger to listen to the event channel.

Maximize Scalability with Asynchronous Decoupling

CDC keeps the main transaction lightweight by offloading processing from synchronous triggers.

AI Demo

*Use Agentforce Vibes 2.0 to help Design for
Scale*

Governance - Documentation & Naming

Flow Documentation

- Apply to individual elements (Get Records, Action, Transform), not just at flow level
- Describe purpose and trigger criteria
- ***Critical for explaining complex logic in invocable actions and subflows***

Apex Comments & Metadata

- **Explain the "why"** behind logic, not just the "what"
- Ensure custom metadata records have **human-readable description** fields explaining what each handler class does and when it runs.

Automated Quality Checks

Use **Salesforce Code Analyzer** to catch pipeline inefficiencies:

- Inefficient Get Records inside a Flow loop
- SOQL queries inside an Apex for loop

Use AI for Documenting Legacy Automation!

Governance - DevOps

Everything is Metadata

Both Apex classes and Salesforce Flows are metadata representing business logic, requiring strict versioning.

Audit Trails & Collaboration

Source control acts as a "single source of truth" for parallel work and provides invaluable audit trails via commit messages.

Regression Prevention

Automated Apex tests ensure new Flow versions don't break existing logic, maintaining stability as automation density grows.

Unified Tooling

DevOps Center provides a unified interface for admins and developers to choreograph source-control development steps.

Leverage Source Control for scalable, reliable deployments!

What did we learn?

Scaling your Salesforce organization requires a fundamental shift in perspective. Focus on these key strategies:

Evaluate Automation Density

Analyze the quantity, volume, and sprawl of your automations to determine the appropriate balance between Flow and Apex.

Architect for Efficiency

Adhere to the "One Entry Point" rule and prioritize decoupled asynchronous processing for high-volume scenarios.

Treat Automation as Metadata

Manage every aspect of your automation—whether built with clicks or code—as versionable metadata.

Utilize AI for Governance

Implement tools like Agentforce Vibes and the Salesforce Code Analyzer to enforce standards and documentation.

Master the balance of Flow, Apex, and AI for a truly scalable organization!

Additional Learning

Salesforce Architecture

- Salesforce Record Triggered Automation: Apex or Flow? (<http://sfdc.co/b7q9Fm>)
- Record Triggered Automation – Decision Guide (<http://sfdc.co/b9SMZ>)

Trailhead

- Record Triggered Flows – Trailhead (<http://sfdc.co/TvhRB>)
- Change Data Capture Basics – Trailhead (<http://sfdc.co/oHTIC>)

Thank you!

Questions?